DOI: 10.1111/mice.12595

RESEARCH ARTICLE

COMPUTER-AIDED CIVIL AND INFRASTRUCTURE ENGINEERING WILEY

A generative adversarial network for travel times imputation using trajectory data

Kunpeng Zhang[1] | **Zhengbing He**[2] | **Liang Zheng**[3] | **Liang Zhao**[1] | **Lan Wu**[1]

[1] College of Electrical Engineering, Henan University of Technology, Zhengzhou, China

[2] Beijing Key Laboratory of Traffic Engineering, Beijing University of Technology, Beijing, China

[3] School of Traffic and Transportation Engineering, Central South University, Changsha, China

Correspondence
Liang Zheng, School of Traffic and Transportation Engineering, Central South University, Changsha, 410075, China.
Email: zhengliang@csu.edu.cn

Funding information
National Key R&D Program of China, Grant/Award Number: 2018YFB1601300; National Natural Science Foundation of China, Grant/Award Numbers: 71871227, 71871010, 61473114, 61973103; Innovation Driven Plan of Central South University, Grant/Award Number: 20180016040002; Fundamental Research Funds for the Henan Provincial Colleges and Universities in Henan University of Technology, Grant/Award Number: 2018RCJH16

Abstract

Knowledge of travel times serves an important role in traffic control and management. As an increasingly popular data source, vehicle trajectories can provide large-scale travel time information. However, real-world travel time information extracted from sparse or low-resolution trajectory data often contains missing data that need to be imputed for further traffic analysis. Thus, this study proposes a travel times imputation generative adversarial network (TTI-GAN) for travel times imputation. Considering the network-wide spatiotemporal correlations, the TTI-GAN can generate travel times for links without sufficient observations by modeling travel time distributions (TTDs) for links with rich data. Then, numerical experiments are carried out with trajectory data from Didi Chuxing. The results show that the TTI-GAN can well estimate link TTDs and performs better than other counterparts in imputing mean travel times under various data missing rates.

1 | INTRODUCTION

Complete and accurate traffic data play a vital role in monitoring traffic conditions and evaluating system performances (Adeli & Ghosh-Dastidar, 2004; Adeli & Karim, 2005; Jiang & Adeli, 2004b; Vlahogianni, Karlaftis, & Golias, 2014). In practice, however, the missing data arises almost inevitably for a multitude of reasons, such as, malfunctions of hardware or software or communication network errors. This could result in difficulties in traffic control and management (Jiang & Adeli, 2004a; Karim & Adeli, 2003). Thus, the missing data have to be addressed properly by data imputation (Bae et al., 2018). In general, traffic data imputation is done to estimate the missing traffic data. With real-world traffic data collected from detectors, a wide range of methods have been explored to investigate this problem (Laña, Olabarrieta, Vélez, & Del Ser, 2018; Tang, Zhang, Wang, Wang, & Liu, 2015). These methods can be classified into three categories: prediction, interpolation, and statistical learning (Y. Li, Li, & Li, 2014).

Prediction methods aim to establish a mapping relationship between historical observations and unobserved data at the same location (L. Li, Su, Zhang, Lin, & Li, 2015). Well-known prediction models include autoregressive

integrated moving average (ARIMA) (Zhong, Lingras, & Sharma, 2004), Bayesian model (X. Chen, He, & Sun, 2019), and neural network (L. Li, Zhang, Wang, & Ran, 2018; Van Lint, Hoogendoorn, & van Zuylen, 2005). However, when to predict the current missing data point, the prediction models cannot utilize future traffic information, which makes the imputation less effective. Moreover, when a series of consecutive data is missed, the prediction models are unable to achieve the desired performance (Shang, Yang, Gao, & Tan, 2018). So, the current missing data point cannot be predicted accurately without a series of neighboring data from the past.

Interpolation methods fill missing data points by averaging previous data with a temporal-neighboring strategy or a pattern-similar strategy (Y. Li et al., 2014). That is, each missing data point is estimated through the average or the weighted average of neighboring data (Van Lint et al., 2005). The typical temporal-neighboring interpolation method is to estimate the missing data with previous data collected from the same detector at the same time (Allison, 2001). The classical pattern-similar interpolation method is the k-nearest neighbors (k-NN) model (Haworth & Cheng, 2012), which estimates the missing data with the neighboring points determined by an appropriate distance metric. However, the effectiveness of interpolation methods strongly depends on the assumption that neighboring traffic states are similar to each other. In addition, the performance of interpolation methods is not reliable when the neighboring traffic data are unavailable.

Statistical learning methods attempt to capture the statistic features in observed data, and then impute the missed data by a probability distribution built with the observed data (Laña et al., 2018). The typical Markov Chain Monte Carlo (MCMC)-based method (Steinmetz & Brownstone, 2005) treats a missing data value as a parameter of the model. The principal component analysis (PCA)-based methods are also widely utilized in traffic data imputation (L. Li, Li, & Li, 2013; Qu, Li, Zhang, & Hu, 2009), in which the PCA helps to remove the relatively trivial details of traffic data. However, the complexity of urban road traffic systems makes it significantly problematic to explicitly estimate travel times by the statistical learning methods due to their limited learning capability. Besides, the assumptions made in these methods are not always appropriate in practice.

Fortunately, machine learning has gained increasing interest as a way to address many problems (Rafiei & Adeli, 2017, 2018a; Rafiei, Khushefati, Demirboga, & Adeli, 2017; Zheng et al., 2018). As an important branch of machine learning, deep learning has been increasingly applied in traffic data imputation due to its state-of-the-art results in various domains (Duan, Lv, Liu, & Wang, 2016; Ku, Jagadeesh, Prakash, & Srikanthan, 2016; Lv, Duan, Kang, Li, & Wang, 2014; Rafiei & Adeli, 2018b; Zhang, Liu, & Zheng, 2019; Zhang, Zheng, Liu, & Jia, 2019; Zhuang, Ke, & Wang, 2019). Specifically, with the consideration of spatiotemporal correlations, Duan et al. (2016) presented a denoising stacked autoencoder (DSAE) model to carry out traffic-flow data imputation with the highest data missing rate of 50%. Zhuang et al. (2019) transformed the raw traffic data into spatiotemporal images and then proposed a convolutional neural network (CNN)-based context encoder to reconstruct the images with the data missing rate from 5 to 50%. However, most existing deep learning methods are functioned as time series models, in which historical observations (e.g., $x_1, x_2, \ldots, x_t$) are taken sequentially to predict or estimate the future value (e.g., $x_{(t+1)}$). This practice does not agree with the travel time distribution (TTD) estimation, which is a critical part in this study. That is, without an explicit sequence, the various observations during a time interval are treated equally to estimate a series of future travel times by modeling the distribution of historical observations.

Moreover, some scholars creatively introduce generative adversarial networks (GAN) to estimate traffic states with data from stationary loop detectors (Y. Chen, Lv, & Wang, 2019; Liang, Cui, Tian, Chen, & Wang, 2018; Lin, Dai, Li, & Wang, 2018; Lv, Chen, Li, & Wang, 2018). As an advanced deep learning paradigm, GAN is found to be capable of imputing missing traffic data well. Specifically, with traffic data from several loop detectors installed along a freeway, Liang et al. (2018) proposed a deep generative adversarial architecture (GAA) to estimate the missing traffic flow and traffic density values. Y. Chen et al. (2019) proposed a GAN-based approach to impute traffic flow data with a representation loss to narrow the gap between generative data and real data. However, this study predicted the missed traffic flow values of one detector station by using its own records only. Obviously, these GAN-based methods impute the missed values with traffic data from one or several detectors, which fails to consider spatiotemporal correlations at a network level. Meanwhile, they only impute a single data point for a certain detector during a time interval, which does not satisfy the need to impute multiple data points. Thus, it is still at an early stage, although the GAN-based method provides a novel way for traffic data imputation.

In addition, the aforementioned methods are proposed to address the data missing problem for stationary detectors, which typically have low deployment rates. In this study, the global positioning system (GPS)-based vehicle trajectories are used to extract network-wide traffic states, due to their increasing availability (Bie, Xiong, Yan, Qu, 2020; He, Zheng, Chen, & Guan, 2017; He, Qi, Lu, & Chen, 2019; Liu, Liu, Vu, & Lyu, 2019). However, the vehicle trajectories are typically opportunistic and sparse because

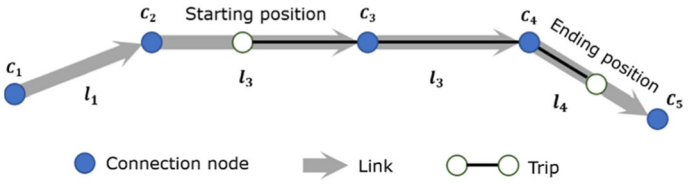


FIGURE 1 Illustration of road network components

of the low penetration rate of GPS vehicles in the road network. This may result in an inaccurate estimation of network-wide traffic states because very few or no samples are observed possibly for many links. To address this problem, this study proposes a travel times imputation generative adversarial network (TTI-GAN) to impute network-wide travel times. Meanwhile, the Gaussian mixture model (GMM) (Williams & Rasmussen, 1996) and Skip–Gram neural network model (Mikolov, Sutskever, Chen, Corrado, & Dean, 2013) are incorporated to capture the network-wide spatiotemporal correlations, which helps the proposed TTI-GAN explicitly model link TTDs and then generate link travel times accordingly. In particular, the link TTDs will be estimated under various data missing rates by the TTI-GAN with an advanced network structure.

The remainder of this article is organized as follows. Section 2 presents some preliminaries and introduces basic concepts. Section 3 proposes the TTD estimation framework, along with the detailed description of the travel times imputation method. Numerical experiments are conducted in Section 4. Section 5 draws conclusions.

2 | PRELIMINARIES

2.1 | Basic definitions

Road network components are defined to facilitate their denotations (Figure 1). A road network is denoted by $R = (L, C)$, where L and C are the sets of links and nodes, respectively. There are two connection nodes for each link, that is, the starting node (l_{start}) and the ending node (l_{end}), which also indicate the traffic flow direction. Thus, a link is defined as $l = [l_{\text{start}}, l_{\text{end}}]$. With the time and space information, the word description of link l is given as $[l, day, time, l_{\text{start}}, l_{\text{end}}]$, in which l is the link ID, *day* is the day of the week (e.g., Wednesday), and *time* is the time of the day (e.g., 08:00:00). The word descriptions of links reflect the evolution of the road network across time and space. Take link 1 as an example, on Wednesday morning at eight o'clock, the word description is *[1, Wednesday, 08:00:00, 100, 101]*.

A trip consists of a series of successive links, and may start and end at an interior position of links. Each GPS trajectory point is expressed as $G = (d, trip, t, [\lambda, \varphi])$, where d is an anonymized driver ID, *trip* is an anonymized order ID (i.e., trip ID), t is the timestamp (e.g., 2016-11-01 08:00:00), and $[\lambda, \varphi]$ is the GPS coordinate position at timestamp t.

2.2 | Map matching and travel time calculation

During the data preprocessing, GPS trajectory points of *trip i* are first projected onto the most likely links $l_1, l_2, \ldots, l_k, \ldots, l_K$ in a road network, where l_k is the kth link of this trip. For more details about the map matching method refer to Hunter, Abbeel, and Bayen (2013). This study assumes that the GPS trajectory observations have been properly allocated to the corresponding road links. Moreover, let O_n represent the nth GPS trajectory observation on link l_k of *trip i*. Two successive observations (i.e., O_n and O_{n+1}) are set as a pair of GPS trajectory observations p, whose geographical distance is denoted by $d_p(O_n, O_{n+1})$.

The travel time from O_n to O_{n+1} is $t_p(O_n, O_{n+1}) = t_{n+1} - t_n$, and then the average travel speed S_k^i on link l_k of *trip i* is computed by:

$$S_k^i = \sum_{p=1}^{P} d_p(G_n, G_{n+1}) / \sum_{p=1}^{P} t_p(O_n, O_{n+1}) \tag{1}$$

where P is the number of pairs of GPS trajectory observations on link l_k of *trip i*.

The travel time T_k^i on link l_k of *trip i* is calculated as:

$$T_k^i = D_k / S_k^i \tag{2}$$

where D_k is the length of link l_k.

3 | METHODOLOGY

The methodology framework includes four components. First, the network structure of the TTI-GAN model is detailed. Second, the GMM is introduced to cluster the TTDs of links by exploring their traffic states, based on which the best fine-tuned TTI-GAN model is saved for each cluster. Third, the word descriptions of a road network are encoded by a Skip–Gram neural network model to obtain the semantic vectors, which help the TTI-GAN model capture the spatiotemporal correlations between

link TTDs. Fourth, the link travel times, the cluster labels, and the semantic vectors are respectively separated into three parts, that is, training, validation, and estimation data sets. Specifically, 80% of the data are used to train the TTI-GAN model, 10% of the data to carry out the random cross-validation, which is to avoid biased estimations and obtain the best TTI-GAN model for each cluster, and the remaining 10% of the data to check the estimation performance. In the following section, details about the original GAN and the TTI-GAN will be elaborated.

3.1 | Generative adversarial network

The GAN is an example of generative models (Goodfellow et al., 2014), and its general structure consists of two models that are alternatively trained to compete with each other. The generator G is optimized to represent the true data distribution p_{real}, by generating samples that are difficult for the discriminator D to differentiate from real samples. Overall, the training procedure is similar to a two-player min-max game with the objective function as:

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{real}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1-D(G(z)))] \quad (3)$$

where x is a real image (real travel times in the context of the travel times imputation) from the true data distribution p_{real}, and z is a noise vector sampled from the distribution p_z (e.g., a uniform or normal distribution). In practice, the generator G is modified to maximize $\log(D(G(z)))$ instead of minimizing $\log(1-D(G(z)))$ to mitigate the gradient vanishing (Goodfellow et al., 2014). The training procedure of the original GAN model is given as Algorithm 1.

Algorithm 1 The original GAN model training

for number of training iterations **do**
  Initialize hyperparameters;
  Feed the noise vector z from p_z into D;
  Feed x from the true data distribution p_{real} into D;
  Maximize $V(D, G)$, update D;
  Feed the noise vector z from p_z into G;
  Minimize $V(D, G)$, update G.
end for
  return G

3.2 | Travel times imputation GAN

This section will elaborate the TTI-GAN model, which can generate travel times for links without sufficient observations by modeling TTDs of links with rich data. The basic idea is to discover the fundamental laws that govern the physics of network-wide traffic, by training the TTI-GAN model. So, the travel time information and the auxiliary vectors that contain the road network information should be available to the TTI-GAN model. Here, a set of hidden codes (i.e., auxiliary vectors) with mutual information (MI) is introduced, which has shown effectiveness in the information maximizing generative adversarial network (InfoGAN) (X. Chen et al., 2016). Figure 2 describes the general structure of the TTI-GAN model. The key difference of the TTI-GAN model from the original GAN model is the introduction of a recognition layer R apart from the discriminator layer D_G in the discriminator network, which helps to learn the interpretable representations for latent variables (i.e., properties of travel times). The latent variables cover the link information in time and space, link clusters, and shapes of TTDs. Specifically, along with link clusters and shapes of TTDs, the link information encoded in semantic vectors can help the TTI-GAN model explicitly distinguish links and their spatiotemporal correlations by training the TTI-GAN model with interpretable properties of link travel times. Moreover, the latent variables include the interpretable hidden code c and a source of uninterpretable noises z, which make the generator become $G(z, c)$. To ensure the effectiveness of this hidden code c, information-theoretic regularization is utilized to ensure the high mutual information between c and $G(z, c)$, which is represented by $I(c; G(z, c))$. In other words, the information in the hidden code c should not be lost in the training process. Thus, the TTI-GAN model attempts to solve the following information regularized min-max game:

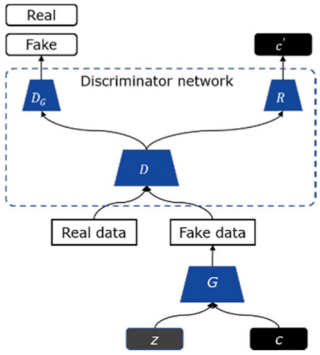


FIGURE 2 General structure of the TTI-GAN model

$$\min_{G,Q} \max_D V_{\text{TTI-GAN}}(D,G,Q) = V_{\text{GAN}}(D,G) - \lambda \cdot L_{\text{Info}}(G,Q) \quad (4)$$

$$I(c;G(z,\ c)) \geq L_{\text{Info}}(G,Q) = \mathbb{E}_{x\sim G(z,c)}[\mathbb{E}_{c'\sim P(c,x)}[\log Q(c'|x)]] + H(c) \quad (5)$$

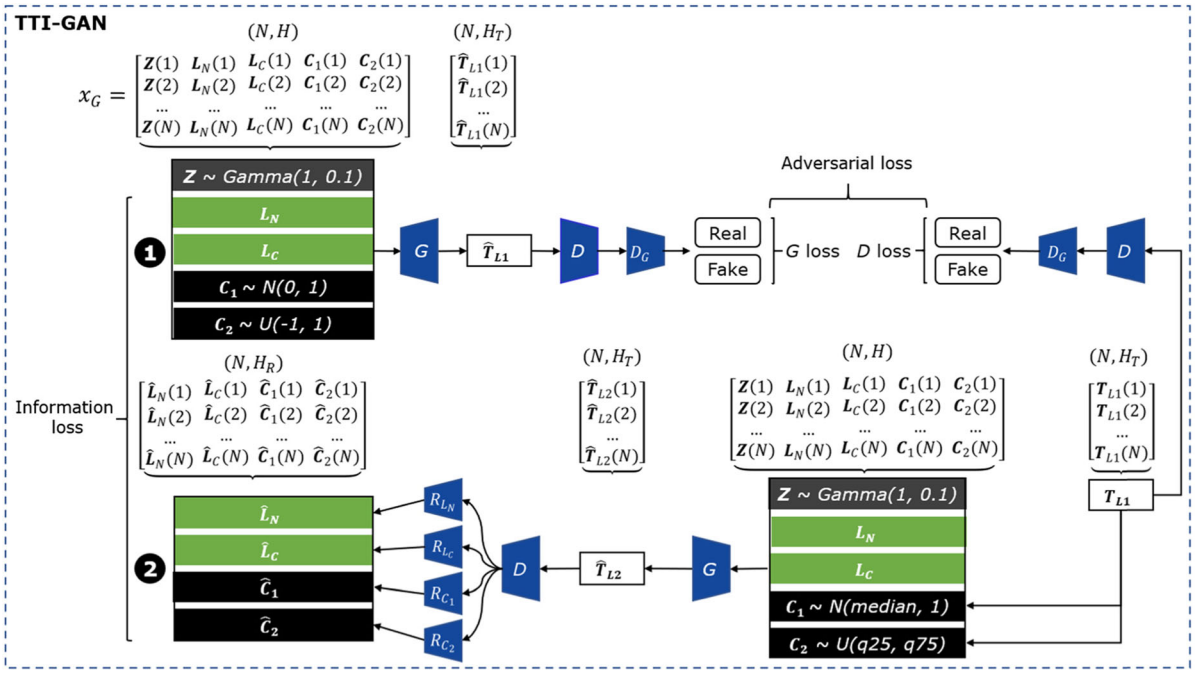


FIGURE 3 Internal structure of the TTI-GAN model. Note: N is the batch size of training data; H is the sum of the dimensions of $\boldsymbol{Z}$, $\boldsymbol{L}_N$, $\boldsymbol{L}_C$, $\boldsymbol{C}_1$, and $\boldsymbol{C}_2$; H_T is the dimension of $\boldsymbol{T}_{L1}$; and H_R is the sum of the dimensions of $\boldsymbol{L}_N$, $\boldsymbol{L}_C$, $\boldsymbol{C}_1$, and $\boldsymbol{C}_2$

where Q is an auxiliary distribution parameterized by the recognition layer R to approximate the posterior $P_G(c|x)$. c' denotes the the synthetic sample corresponding to c. λ is the weighting coefficient and $L_{\text{Info}}(G, Q)$ represents the lower bound of MI $I(c; G(z, c))$. As shown in Equation (5), the lower bound becomes tight as the auxiliary distribution Q approximates the true posterior $P_G(c|x)$ and $\mathbb{E}_{x\sim G(z,c)}[\mathbb{E}_{c'\sim P(c,x)}[\log Q(c'|x)]]$ reduces to 0. Then, the maximal MI between the hidden code c and the generator G is achieved when the lower bound reaches its maximum (i.e., $L_{\text{Info}}(G, Q) = H(c)$).

With the introduction of the hidden codes and MI, the TTI-GAN model is capable of effectively controlling the characteristics of generated travel times according to the input of the generator G. Whereas, the input of the generator G covers a noise vector for improving the generalization, the semantic vectors that contain time and space information of the road network, the cluster labels indicating the similarity of travel time patterns, and other auxiliary vectors representing the properties of link travel times. With the theoretical support proved by InfoGAN, the training stabilization of the TTI-GAN model becomes necessary to achieve desirable performance. Thus, the deep structure of deep convolutional GAN (DCGAN) (Radford, Metz, & Chintala, 2015) and the training strategy of Wasserstein GAN (WGAN) (Gulrajani, Ahmed, Arjovsky, Dumoulin, & Courville, 2017) are introduced to guarantee training stabilization. The detailed implementation is described in the following.

The internal structure of the TTI-GAN model is illustrated in Figure 3, which contains two steps to alternatively refine the generator G and the discriminator network with two kinds of loss, that is, the adversarial loss (i.e., $V_{\text{GAN}}(D, G)$ in Equation (4)) and the information loss (i.e., $L_{\text{Info}}(G, Q)$ in Equation (4)). Moreover, Figure 4 illustrates the detailed network nodes of the generator G and the discriminator network, which are designed by modifying DCGAN to obtain the stable training process of the TTI-GAN model. Meanwhile, this structure applies one-dimensional (abbreviated as *1d*) layers to handle the low-dimensional task. The hyperparameters of the generator G and the discriminator network are listed in Table A1 of the Appendix.

To facilitate the description of the TTI-GAN model, let $\boldsymbol{Z}$ denote the noise vector from a gamma distribution, which has been found to be useful in improving the performance of the TTI-GAN model. $\boldsymbol{L}_N$ represents the semantic vectors from the Skip–Gram neural network model. $\boldsymbol{L}_C$ denotes the cluster labels of link travel times. $\boldsymbol{C}_1$ and $\boldsymbol{C}_2$ are introduced as two kinds of hidden codes, which are from a normal distribution and a uniform distribution, respectively. $\boldsymbol{T}_{L1}$ represents the true link travel times, in which $\boldsymbol{T}_{L1}(i, t)$ $(i = 1, 2, \ldots, I)$ is the ith travel time (the travel time of the ith vehicle) from a link during a time interval t. I is the sample size of link travel times. $\boldsymbol{L}_N(i)$ and $\boldsymbol{L}_C(i)$ are the semantic vectors and cluster labels of $\boldsymbol{T}_{L1}(i, t)$, respectively. In the TTI-GAN model, the input of the generator G includes $\boldsymbol{Z}$, $\boldsymbol{L}_N$, $\boldsymbol{L}_C$, $\boldsymbol{C}_1$, and $\boldsymbol{C}_2$. The

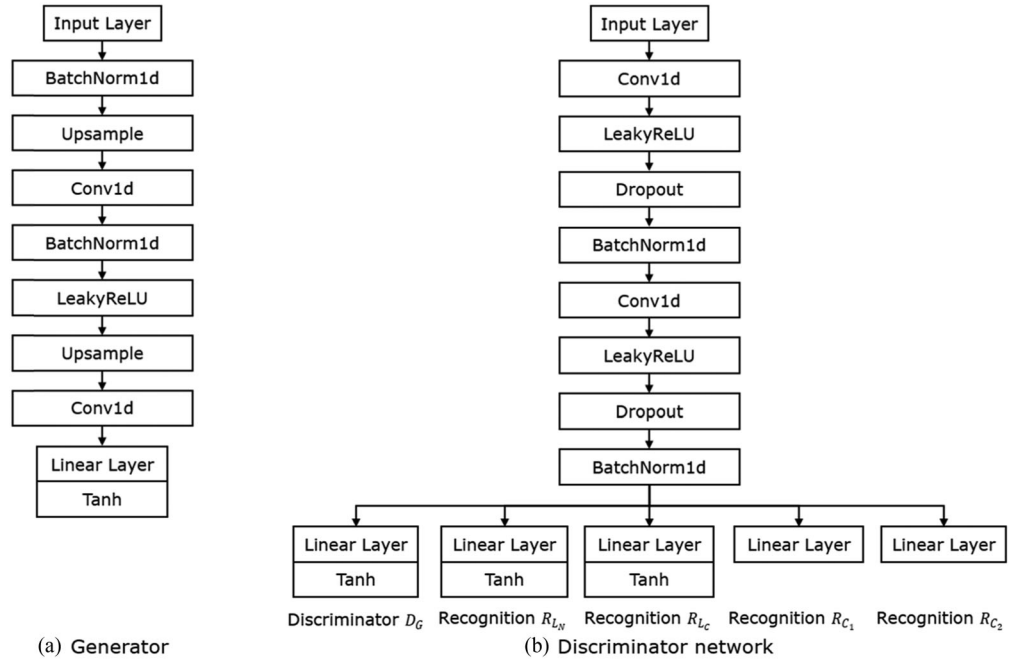


FIGURE 4 Internal structures of the generator G and the discriminator network

following will elaborate two critical steps, that is, Step 1 and Step 2.

Step 1: Minimize the Adversarial loss $\boldsymbol{V_{GAN}}(\boldsymbol{D}, \boldsymbol{G})$

To generate travel times $\hat{\boldsymbol{T}}_{L1}$ for links, $\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1$, and $\boldsymbol{C}_2$ are concatenated as the input of the generator G, that is, $x_G = \text{concatenate}(\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2)$, cf., Figure 3. The shape of x_G is $(N,\ H)$, where N is the batch size of training data in an iteration and H is the sum of the dimensions of $\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1$, and $\boldsymbol{C}_2$. Meanwhile, a linear neural network layer is adopted in the input layer, which is formulated as:

$$y_{G1} = w_G x_G + b_G \tag{6}$$

where w_G and b_G are weights and biases.

To accelerate the training process of the TTI-GAN model, a *BatchNorm1d* layer is utilized to normalize y_{G1} (Figure 4a), which is calculated by:

$$y_{G2} = \frac{y_{G1} - \mu_{G1}}{\sqrt{\sigma_{G1}^2 + \epsilon_{G1}}} \gamma_{G1} + \beta_{G1} \tag{7}$$

where μ_{G1} and σ_{G1} denote the mean and standard deviation of y_{G1}, respectively. γ_{G1} and β_{G1} are the parameters to be learned by the generator G, and ϵ_{G1} is a constant.

After that, the results of the *BatchNorm1d* layer (i.e., y_{G2}) are resampled by the nearest neighbor interpolation in the *Upsample* layer so as to increase the sample rates. The results are denoted as y_{G3}. To capture the local and network-wide spatiotemporal correlations hidden in link travel times, a generator block is utilized in the generator G. It consists of a convolution layer (i.e., *Conv1d* in Figure 4a), a *BatchNorm1d* layer, an activation layer with a leaky rectified linear unit (i.e., *LeakyReLU* in Figure 4a), and an *Upsample* layer. As the main component of the generator G, the generator block is formulized as:

$$y_{G4} = \sum_{m=1}^{M} f_G(m)^* y_{G3} \tag{8}$$

$$y_{G5} = \Phi_{LReLU}\left(\frac{y_{G4} - \mu_{G2}}{\sqrt{\sigma_{G2}^2 + \epsilon_{G2}}} \gamma_{G2} + \beta_{G2}\right) \tag{9}$$

where $f_G(m)$ represents the mth filter of the *Conv1d* layer. The operator * expresses the convolution. μ_{G2} and σ_{G2} are the mean and standard deviation of y_{G4}, respectively. γ_{G2} and β_{G2} are parameters to be learned by the generator G, and ϵ_{G2} is a constant. Φ_{LReLU} denotes the nonlinearity activation operation in the *LeakyReLU* layer. The result from the generator block (i.e., y_{G5}) is fed into a convolution layer. Then, the result of the convolution layer is fed into the output layer of the generator G with a Tanh activation operation, which results in the set of travel times $\hat{\boldsymbol{T}}_{L1}$.

On the other hand, the generated travel times (i.e., $\hat{\boldsymbol{T}}_{L1}$) and real travel times (i.e., $\boldsymbol{T}_{L1}$) are alternatively fed into the discriminator network so as to improve its capability of

distinguishing fake travel times and real ones. When $\hat{\boldsymbol{T}}_{L1}$ is treated as the input (i.e., x_D), a linear neural network layer is adopted as the input layer of the discriminator network, and y_{D1} is the output. Then, two discriminator blocks are stacked between the input and output layers to enhance the discrimination ability of the discriminator network, cf., Figure 4b. With y_{D1} as the input, the computational procedures in the first discriminator block are given as:

$$y_{D2} = \sum_{m=1}^{M} f_D(m)^* y_{D1} \tag{10}$$

$$y_{D3} = \Phi_{LReLU}(y_{D2}) \cdot r \tag{11}$$

$$y_{D4} = \frac{y_{D3} - \mu_D}{\sqrt{\sigma_D^2 + \epsilon_D}} \gamma_D + \beta_D \tag{12}$$

where $f_D(m)$ represents the mth filter of the *Conv1d* layer in the discriminator block. Equation (11) formulates the computational procedures of a *LeakyReLU* layer and a dropout layer, where the operator $\cdot$ denotes an element-wise product. $r \sim Bernoulli(p)$ is a vector of independent Bernoulli random variables, and each variable has the probability p of being 1. μ_D and σ_D are the mean and standard deviation of y_{D3}, respectively. γ_D and β_D are the parameters to be learned by the discriminator network, and ϵ_D is a constant. y_{D4} from the batch normalization layer (i.e., *BatchNorm1d*) of the first discriminator block is then fed into another discriminator block, whose computational procedures are repeated by Equations (10–12).

To evaluate the quality of the generated travel times $\hat{\boldsymbol{T}}_{L1}$, the discriminator network introduces a scoring mechanism. Therefore, the result from the last discriminator block is then fed into the discriminator layer D_G. As a linear layer, D_G outputs a set of scores $\hat{\boldsymbol{S}}$ for $\hat{\boldsymbol{T}}_{L1}$, which represents the quality of the generator G. Let the nth component of $\hat{\boldsymbol{S}}$ (i.e., $\hat{s}_n$) denote the score for the nth set of generated travel times, that is, $\hat{\boldsymbol{T}}_{L1}(n)$ $(n = 1, 2, \ldots, N)$, where N is the batch size of training data in an iteration. The score $\hat{s}_n$ ranges from 0 to 1. If $\hat{\boldsymbol{T}}_{L1}(n)$ is the same as the real one (i.e., $\boldsymbol{T}_{L1}(n)$), $\hat{s}_n$ equals to 1. This means the generated travel times by the generator G are indistinguishable from the real ones, and the generator G is quite qualified to represent the true TTDs of the links. To optimize the generator G, the TTI-GAN model is trained to minimize the difference between $\hat{\boldsymbol{S}}$ and 1, that is, the G loss in Figure 3, which is calculated by the following mean square error:

$$L_G = \frac{1}{N} \sum_{n=1}^{N} (1 - \hat{s}_n)^2 \tag{13}$$

The discriminator network takes a set of fake travel times (i.e., $\hat{\boldsymbol{T}}_{L1}$) or real travel times (i.e., $\boldsymbol{T}_{L1}$) from links as the input so as to improve its skill to differentiate the real travel times from the fake ones. That is, the discriminator network is optimized to obtain high scores $\boldsymbol{S}$ (close to 1) with $\boldsymbol{T}_{L1}$ as the input, while to pursue low score $\hat{\boldsymbol{S}}$ (close to 0) with $\hat{\boldsymbol{T}}_{T1}$ as the input. This objective can be achieved by minimizing the D loss c.f., Figure 3, which is formulated as:

$$L_D = \frac{1}{N} \sum_{n=1}^{N} (1 - s_n)^2 + \frac{1}{N} \sum_{n=1}^{N} (0 - \hat{s}_n)^2 \tag{14}$$

where s_n is the score for the nth set of real travel times, that is, $\boldsymbol{T}_{L1}(n)$.

Therefore, the adversarial loss $V_{\text{GAN}}(D, G)$ consists of two components: L_G and L_D, based on which the discriminator network will be updated so as to enhance its skill.

Step 2: Minimize the Information loss $\boldsymbol{L}_{\text{Info}}(\boldsymbol{G}, \boldsymbol{Q})$

In the TTI-GAN model, to attain the maximal MI between $\boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2$, and $G(\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2)$, the following procedures are introduced with the information loss $L_{\text{Info}}(G, Q)$ (c.f., Figure 3) as the optimization objective. In this step, $\boldsymbol{C}_1$ and $\boldsymbol{C}_2$ in the inputs of Step 1 are replaced by the normal distribution with the median of $\boldsymbol{T}_{L1}$ as the mean and the uniform distribution with the 25th percentile and 75th percentile of $\boldsymbol{T}_{L1}$ as the lower and upper boundary. Associated with the $\boldsymbol{Z}, \boldsymbol{L}_N$, and $\boldsymbol{L}_C$ in Step 1, the updated $\boldsymbol{C}_1$ and $\boldsymbol{C}_2$ are taken as the input of the generator G to generate $\hat{\boldsymbol{T}}_{L2}$. Then, $\hat{\boldsymbol{T}}_{L2}$ is taken as the input of the discriminator network, whose output is referred to as y_R. Then, with y_R as the input, the recognition layer R (i.e., R_{L_N}, R_{L_C}, R_{C_1}, and R_{C_2}, c.f., Figure 4b) tries to output $\hat{\boldsymbol{L}}_N, \hat{\boldsymbol{L}}_C, \hat{\boldsymbol{C}}_1$, and $\hat{\boldsymbol{C}}_2$ as the semantic vectors, the cluster labels and two sets of hidden codes, respectively.

The parameters of the generator G and the discriminator network are updated again with the information loss $L_{\text{Info}}(G, Q)$ as the optimization objective. Then, the MI between $\boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2$ and $G(\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2)$ is formulated as:

$$\begin{aligned} I(\boldsymbol{L}_N; G(\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2)) &\geq L_{\text{Info}}(G, Q_{L_N}) \\ &= \mathbb{E}_{x \sim G(\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2)} \left[\mathbb{E}_{c' \sim P(c,x)} \left[\log Q_{L_N}(c'|x) \right] \right] + H(\boldsymbol{L}_N) \end{aligned} \tag{15}$$

$$\begin{aligned} I(\boldsymbol{L}_C; G(\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2)) &\geq L_{\text{Info}}(G, Q_{L_C}) \\ &= \mathbb{E}_{x \sim G(\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2)} \left[\mathbb{E}_{c' \sim P(c,x)} \left[\log Q_{L_C}(c'|x) \right] \right] + H(\boldsymbol{L}_C) \end{aligned} \tag{16}$$

$$\begin{aligned} I(\boldsymbol{C}_1; G(\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2)) &\geq L_{\text{Info}}(G, Q_{C_1}) \\ &= \mathbb{E}_{x \sim G(\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2)} \left[\mathbb{E}_{c' \sim P(c,x)} \left[\log Q_{C_1}(c'|x) \right] \right] + H(\boldsymbol{C}_1) \end{aligned} \tag{17}$$

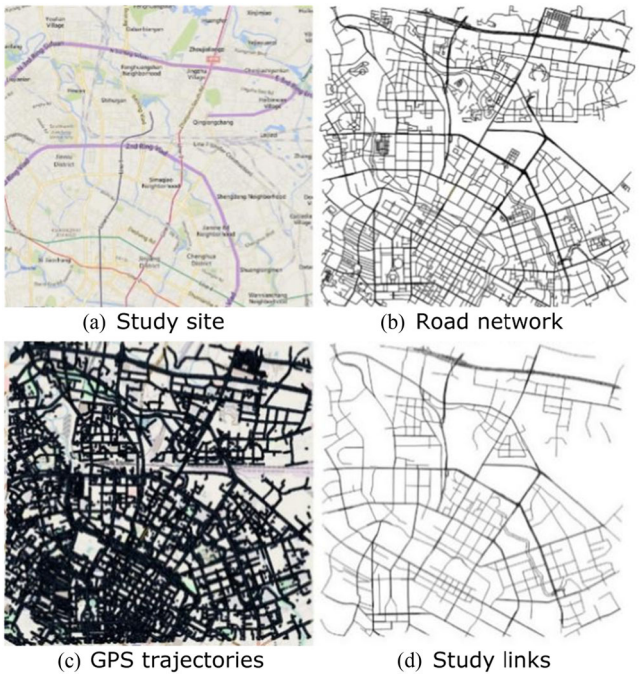


FIGURE 5 Road network of the study site

$$I\left(\boldsymbol{C}_2; G\left(\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2\right)\right) \geq L_{\text{Info}}\left(G, Q_{C_2}\right)$$
$$= \mathbb{E}_{x\sim G(\boldsymbol{Z},\boldsymbol{L}_N,\boldsymbol{L}_C,\boldsymbol{C}_1,\boldsymbol{C}_2)}\left[\mathbb{E}_{c'\sim P(c,x)}\left[\log Q_{C_2}\left(c'|x\right)\right]\right] + H\left(\boldsymbol{C}_2\right) \quad (18)$$

where Q_{L_N}, Q_{L_C}, Q_{C_1}, and Q_{C_2} are the auxiliary distributions parameterized by $R_{L_N}, R_{L_C}, R_{C_1}$, and R_{C_2}, respectively.

Afterward, the information loss $L_{\text{Info}}(G, Q)$ is calculated by:

$$L_{\text{Info}}\left(G, Q\right) = a_1 L_{\text{Info}}\left(G, Q_{L_N}\right) + a_2 L_{\text{Info}}\left(G, Q_{L_C}\right) + a_3 L_{\text{Info}}\left(G, Q_{C_1}\right) + a_4 L_{\text{Info}}\left(G, Q_{C_2}\right) \quad (19)$$

where $a_i,\ i = 1,\ 2,\ 3,\ 4$ denotes the weight of each component of the information loss and can be treated as the hyperparameters to be learned by the TTI-GAN model.

Inspired by the performance of WGAN, the difference between the real semantic vectors (i.e., $\boldsymbol{L}_N$) and the fake ones (i.e., $\hat{\boldsymbol{L}}_N$) is measured by the Wasserstein distance (WD) (Dobrushin, 1970). This benefits the stability of the training process of the TTI-GAN model. The difference between the real cluster labels (i.e., $\boldsymbol{L}_C$) and the fake ones (i.e., $\hat{\boldsymbol{L}}_C$) is calculated by the cross-entropy loss (CL). The differences between the real hidden codes (i.e., $\boldsymbol{C}_1$ and $\boldsymbol{C}_2$) and the fake ones (i.e., $\hat{\boldsymbol{C}}_1$ and $\hat{\boldsymbol{C}}_2$) are measured by WD. So, the information loss $L_{\text{Info}}(G, Q)$ is reformulated as:

$$L_{\text{Info}}\left(G, Q\right) = a_1 d_W\left(\boldsymbol{L}_N, \hat{\boldsymbol{L}}_N\right) + a_2 d_W\left(\boldsymbol{L}_C, \hat{\boldsymbol{L}}_C\right) + a_3 CL\left(\boldsymbol{C}_1, \hat{\boldsymbol{C}}_1\right) + a_4 CL\left(\boldsymbol{C}_2, \hat{\boldsymbol{C}}_2\right) \quad (20)$$

With the help of the information loss $L_{\text{Info}}(G, Q)$, the generator G and the discriminator network can be optimized to retain the information of $\boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1$, and $\boldsymbol{C}_2$ in the training process of the TTI-GAN model. This promises the superior performance of the TTI-GAN model in estimating the link TTDs. The training procedure of the TTI-GAN model is demonstrated in Algorithm 2.

Algorithm 2 The TTI-GAN model training

for number of training iterations **do**
Initialize hyperparameters Feed $\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2$ into the generator G
Generate travel times $\hat{\boldsymbol{T}}_{L1}$ Feed $\hat{\boldsymbol{T}}_{L1}$ into the discriminator network Get $\hat{\boldsymbol{S}}$ for $\hat{\boldsymbol{T}}_{L1}$ Minimize L_G, update the generator G Feed $\hat{\boldsymbol{T}}_{L1}$ and $\boldsymbol{T}_{L1}$ into the discriminator network alternatively; Get $\hat{\boldsymbol{S}}$ for $\hat{\boldsymbol{T}}_{L1}$, get $\boldsymbol{S}$ for $\boldsymbol{T}_{L1}$ Minimize L_D, update the discriminator network
$\boldsymbol{C}_1 \sim N(\text{median}(\boldsymbol{T}_{L1}), 1)$, $\boldsymbol{C}_2 \sim U(q25(\boldsymbol{T}_{L1}),\ q75(\boldsymbol{T}_{L1}))$
Feed $\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2$ into the generator G
Generate travel times $\hat{\boldsymbol{T}}_{L2}$ Feed $\hat{\boldsymbol{T}}_{L2}$ into the discriminator network Get $\hat{\boldsymbol{L}}_N, \hat{\boldsymbol{L}}_C, \hat{\boldsymbol{C}}_1$ and $\hat{\boldsymbol{C}}_2$ from Recognition $R_{L_N}, R_{L_C}, R_{C_1}$ and R_{C_2}, respectively
Minimize $L_{Info}(G, Q)$, update the generator G and the discriminator network
end for
return the generator G according to cluster labels $\boldsymbol{L}_C$.

4 | EXPERIMENTS AND RESULTS

4.1 | Study site selection and data description

The study site is located in the downtown of Chengdu, China (from 104.043° E to 104.130°E in longitude, and from 30.653°N to 30.726 ° N in latitude; see Figure 5). The road network contains 3,986 links. The traffic data are collected from the on-demand service platform of Didi Chuxing recording anonymized driver ID, anonymized order ID (referred to trip ID here), trajectory positions, and timestamps. The trajectory positions are sampled every three seconds. That is, about 35 million trajectory observations exist per day. The data collection period is from November 1 to November 30, 2016.

After the map matching, 2,079 links (Figure 5d) are selected, which cover the arterial roads of the study site and about 85% of the GPS trajectories. Then, the link travel times are obtained by the travel time calculation algorithm in Section 3.2 and partitioned with a time interval of 30 minutes. So, each link obtains 48 groups of travel times in each day of a week. Moreover, the median absolute deviation (MAD) method is used for the outlier identification. A value is considered an outlier if it is outside the range

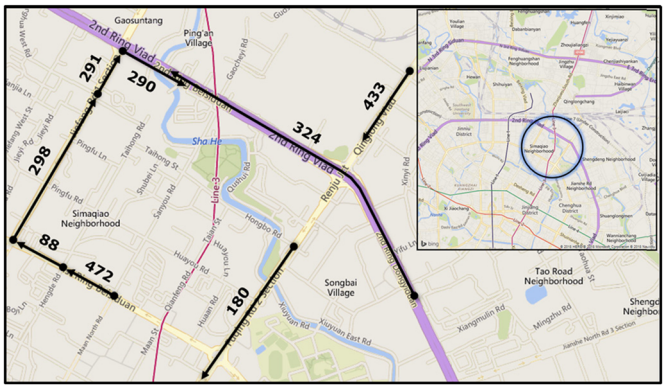


FIGURE 6 Selected links for encoding spatial and temporal correlations by the Skip–Gram neural network model

of the lower and upper bound values determined by the MAD 3-delta criterion (Pearson, 2002). Then, the groups with the number of travel times less than 100 are excluded so as to ensure the sufficient number of travel time samples to train the TTI-GAN model. After that, the travel times of all links are normalized to [-1, 1] by utilizing Max–Min normalization.

4.2 | Road network encoding

Due to the important role of network-wide spatiotemporal correlations in the accurate estimation of link TTDs, the Skip–Gram neural network model is employed to encode the spatial information of the underlying road network structure and the temporal properties of the link travel times in various time intervals. Specifically, the road network information is encoded on the basis of the word descriptions of links (Section 2.1), which however belong to high-dimensional information and cannot be directly processed by a deep learning model. Thus, the Skip–Gram neural network model is introduced to reduce the dimensions of the text corpus and generate low-dimensional semantic vectors, which contain the evolution information of a road network in time and space.

Then, eight links are randomly selected from the study site to demonstrate the encoding results of spatial correlations. The selected links are shown in Figure 6, in which link 472, 88, 298, 291, and 290 are successive links and links 433, 324, and 180 do not have a connection with each other. Each link is described as a word description and all links have the same timestamp (e.g., 08:00 on Wednesday) so as to ensure no temporal diversity in the word descriptions, which are then taken as the input of the Skip–Gram neural network model to provide the semantic vectors. Moreover, the PCA projection shows that the semantic vectors of the five successive links are closely organized, while the unconnected links are scattered in the 2D space, c.f., Figure 7a. This illustrates the ability of the Skip–Gram neural network model to encode the underlying structure of the road network and capture the implicit spatial correlations.

As for temporal correlations, the encoding results of link 290 under various timestamps are shown in Figure 7b. It is observed that the semantic vectors at 00:00 are accurately separated from those at 08:00 in terms of the X axis. The semantic vectors under various days of a week are scattered with a correct order along the Y axis (i.e., Wednesday, Thursday, Saturday, and Sunday). This illustrates that the semantic vectors provided by the Skip–Gram neural network model are able to reflect the distinctiveness of each word description and capture the correlations of link traffic states across time.

To sum up, the Skip–Gram neural network model is able to encode the word descriptions with spatial and temporal characteristics of a road network into the low-dimensional semantic vectors, which can be utilized by a deep learning model. These semantic vectors solve the dimension curse and meanwhile retain the spatial and temporal semantic correlation between the word descriptions. That is, these semantic vectors would contain the evolution information of a road network in time and space, which is indispensable for the TTI-GAN model to distinguish links and their spatiotemporal relationships.

4.3 | Experiment results

This section takes link 290, 433, 324, and 180 (c.f., Figure 6) as an example to validate the performance of the proposed approaches in estimating the link TTDs. We first collect all travel times of the selected links, and then randomly delete travel times from the raw data in all time intervals so as to achieve a certain data missing rate. Then, the WD is used to assess the performance of the TTI-GAN model in estimating the link TTDs. If the WD value is 0, the estimated TTD agrees with the observed one. Statistical measures are also calculated to evaluate the estimation accuracy, including mean, standard deviation (*SD*), skewness, and kurtosis. The Kolmogorov–Smirnov (KS) test is employed to test the null hypothesis that the estimated distribution equals the observed one. Skewness is a measure of the asymmetry of a probability distribution, which appears skewed either to the left (negative skewness) or to the right (positive skewness). Kurtosis is a measure of the flatness of a probability distribution. A negative kurtosis represents a platykurtic distribution, which appears to be flatter (heavy-tailed) than a normal distribution. A positive kurtosis stands for a leptokurtic distribution, which appears to be less flat (light-tailed) than a normal distribution.

FIGURE 7 PCA projection of semantic vectors

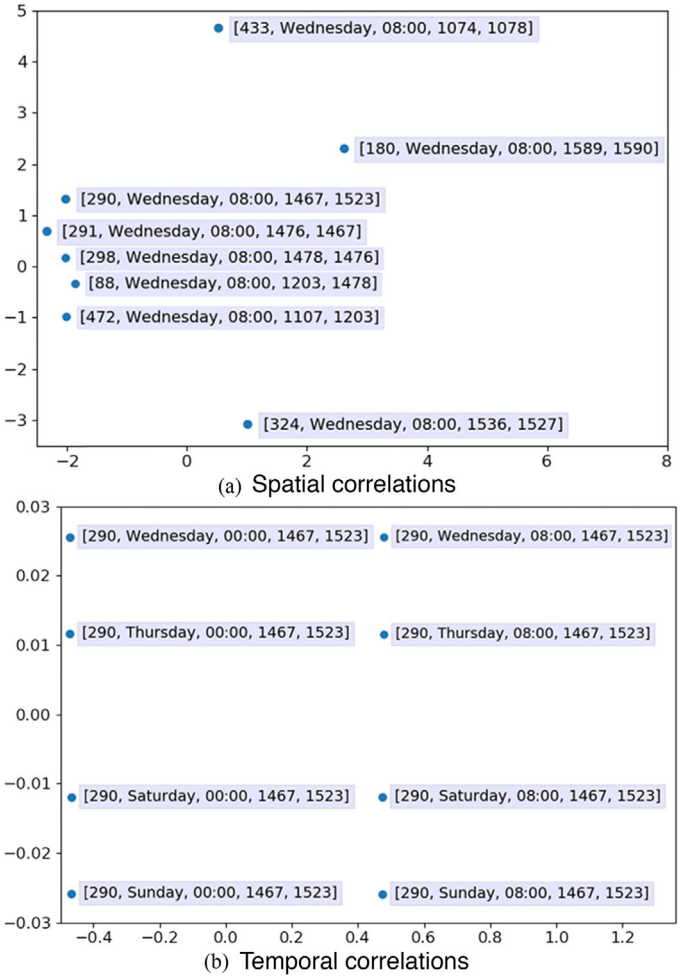
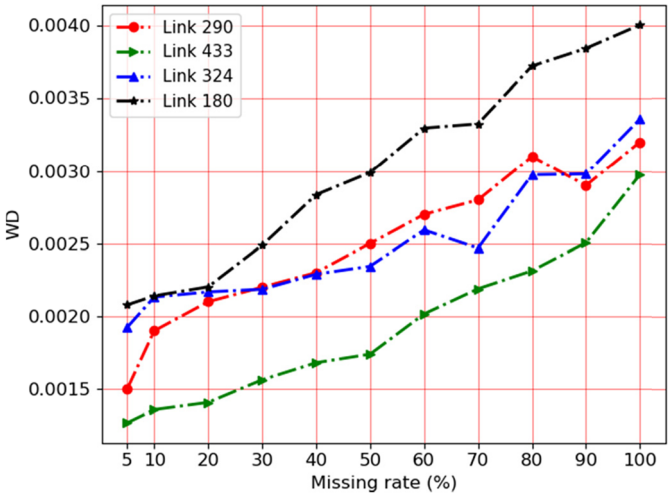


FIGURE 8 WD curves for links 290, 433, 324, and 180



Thus, both skewness and kurtosis can describe the extent to which a distribution differs from a normal distribution.

Under various missing rates, Figure 8 illustrates the WDs for links 290, 433, 324, and 180 during the time interval of 8:00–8:30 on Wednesday. It is observed that with the missing rate increasing from 5 to 100%, the WD between the observed TTD and the estimated one grows gradually for links 290, 433, 324, and 180. This can be

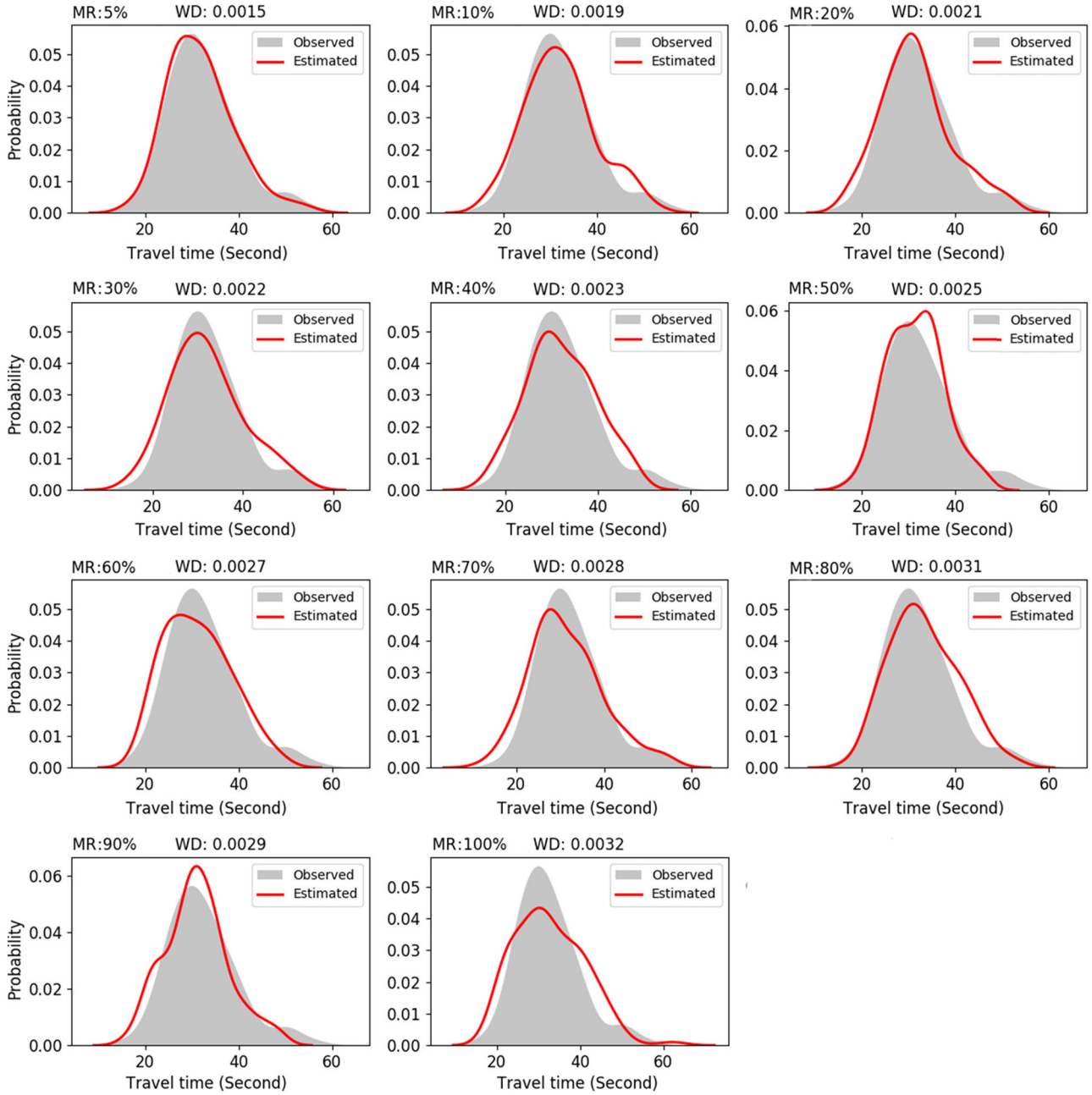


FIGURE 9 TTD estimation for link 290 under various missing rates (MR)

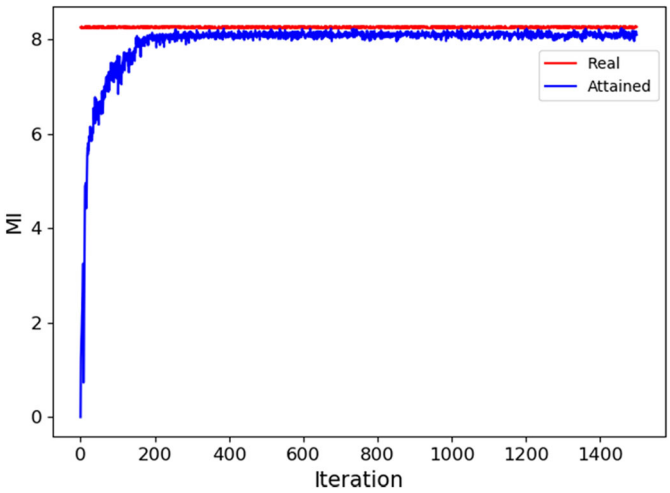
analyzed as follows; as the missing rate increases, the travel time information of the target link vanishes gradually from the training data set, which results in the performance degradation of the TTI-GAN model. This could widen the gap between the observed travel times and the estimated ones, and consequently leads to the increase of the WD.

Taking link 290 as an example, Figure 9 shows the imputation performance of the TTI-GAN model under the missing rate from 5 to 100%. Obviously, as the missing rate increases, the discrepancy between the observed TTD and the estimated one becomes apparent gradually in terms of location, scale, and shape, and the WD raises from 0.0015 to 0.0032. However, the TTI-GAN model is capable to capture the main details of the observed TTD even when the missing rate is larger than 60%. As shown in Table 1, when the missing rate reaches 60%, the TTI-GAN model is still able to generate reasonable travel times and passes the KS test. Moreover, other indices (i.e., mean, *SD*, skewness, and kurtosis) are also approaching to the real values even when the missing rate is larger than 60%. The reason can be analyzed as follows: Although the travel times are missed in varying degrees, the TTI-GAN model can effectively estimate the TTD and then generate reasonable travel

TABLE 1 Performance measures under various missing rates (MR) for link 290

Model	MR (%)	WD	KS	Mean	SD	Skewness	Kurtosis
Observed	–	0.0000	1	32.322 (0.000)	7.368 (0.000)	0.802 (0.000)	3.865 (0.000)
TTI-GAN	5	0.0015	1	31.839 (0.483)	7.045 (0.323)	0.705 (0.097)	3.603 (0.262)
	10	0.0019	1	32.253 (0.069)	7.603 (0.235)	0.406 (0.396)	2.863 (1.002)
	20	0.0021	1	31.614 (0.708)	7.535 (0.167)	0.563 (0.239)	3.119 (0.746)
	30	0.0022	1	32.039 (0.283)	8.086 (0.718)	0.479 (0.323)	2.877 (0.988)
	40	0.0023	1	32.061 (0.261)	7.349 (0.019)	0.146 (0.656)	2.437 (1.428)
	50	0.0025	1	31.527 (0.795)	5.869 (1.499)	0.229 (0.573)	2.735 (1.130)
	60	0.0027	1	31.236 (1.086)	7.030 (0.338)	0.400 (0.402)	2.397 (1.468)
	70	0.0028	0	31.418 (0.904)	8.214 (0.846)	0.579 (0.223)	3.240 (0.625)
	80	0.0031	0	33.310 (0.988)	7.179 (0.189)	0.370 (0.432)	2.607 (1.258)
	90	0.0029	0	31.046 (1.276)	6.748 (0.620)	0.330 (0.472)	3.018 (0.847)
	100	0.0032	0	32.661 (0.339)	8.204 (0.836)	0.542 (0.260)	3.140 (0.725)

FIGURE 10 MI over training iterations



times for the link with few observations with the help of the spatiotemporal correlation of travel times at a network level.

4.4 | Mutual information measurement

This section trains the TTI-GAN model and calculates MI every iteration, which aims to evaluate whether the MI between the latent variables (i.e., link information in time and space [$\boldsymbol{L}_N$], link clusters [$\boldsymbol{L}_C$], and shapes of TTDs [$\boldsymbol{C}_1$and $\boldsymbol{C}_2$]) and $G(\boldsymbol{Z}, \boldsymbol{L}_N, \boldsymbol{L}_C, \boldsymbol{C}_1, \boldsymbol{C}_2)$ can be maximized efficiently. Figure 10 compares the real MI and the MI attained by the TTI-GAN model. It can be found that the attained MI reaches the real one after about 170 training iterations, which shows the lower bound $L_{\text{Info}}(G, Q)$ is quickly maximized to $H(c)$. This means that the TTI-GAN model is able to generate travel times conditioned on the latent variables by encouraging the generator G to maximize the MI with the hidden codes.

4.5 | Model comparison

This section introduces four other counterpart methods to validate the superiority of the TTI-GAN model in imputing travel times. The detailed descriptions are given as follows:

1. ARIMA: As a widely used time series prediction model, ARIMA is able to consider trends, cycles, and nonstationary characteristics of a data set simultaneously. When applying the ARIMA-based imputation method, the historical time series data set is utilized to train this model and then the fine-tuned model is employed to recover the missing data points. If the data points are consecutively missed, the imputed data will be used as known data to estimate the next missing data.
2. k-NN: Weighted k-NN is a well-known nonparametric method. When imputing the missed data, k nearest travel time vectors are selected for the corrupted

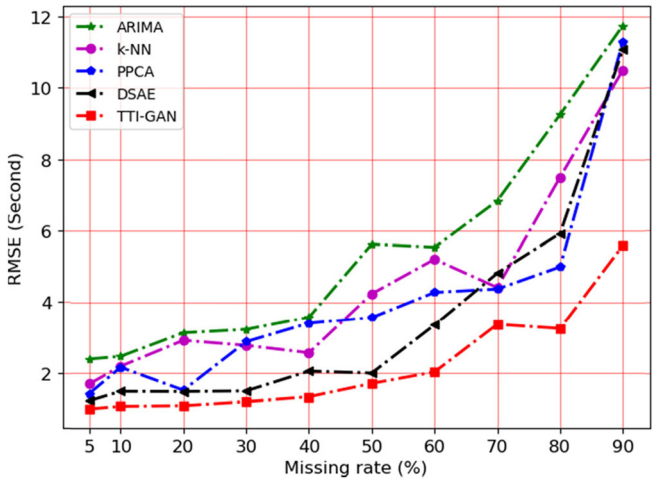


FIGURE 11 Imputation errors under various missing rates

vector, and then the missing entry can be imputed accordingly.

3. PPCA: Probabilistic Principal Component Analysis (PPCA) assumes a model where the observed data depend on the latent variables $\boldsymbol{Y} = Wx + \mu + \varepsilon$. $\boldsymbol{Y}$ denotes observed data, x is a latent variable from a normal distribution, μ avoids the model obtaining a zero mean, and ε represents isotropic noise from a normal distribution. The PPCA method is inspired by Qu et al. (2009).
4. DSAE: The structure of DSAE is similar to that proposed by Duan et al. (2016). This deep learning-based model treats the corrupted vector as travel time data that contain the observed normal data points and missing data points. It transforms data imputation into corrupt data denoising.

The aforementioned four counterparts can only impute traffic data in a determinative pattern, that is, imputing a single data point in a certain time interval. In this context, the travel times generated by the TTI-GAN model will be averaged to obtain the mean travel time for a time interval. All numerical experiments are carried out in a desktop computer equipped with Core i7-7700 central processing unit, 16 GB memory, and a GeForce GTX 1080Ti graphics processing unit. The imputation error is evaluated by the root mean squared error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(Y_i - \hat{Y}_i\right)^2} \tag{21}$$

where Y_i and $\hat{Y}_i$ are the ith ground truth and estimated travel time value, respectively, and N denotes the total number of testing entries.

Figure 11 shows the mean RMSEs of 500 links under various missing rates. These links are selected randomly from the road network. It is found that the TTI-GAN model outperforms the other counterparts under any missing rate. Specifically, as the missing rate increases from 5 to 90%, ARIMA performs worst with RMSE growing from 2.400 to 11.748. The performance of k-NN is slightly better than ARIMA, with RMSE increasing from 1.700 to 10.500. As for PPCA and DSAE, the reasonable performance is achieved when the missing rate is low, but RMSEs of PPCA and DSAE raise from 1.432 to 11.295 and from 1.231 to 11.079, respectively as the missing rate grows. The TTI-GAN model achieves the best performance with RMSE increasing from 1.004 to 5.594. It is also worth noticing that the TTI-GAN model can address the extreme case when the missing rate even reaches 90%. This also validates the robustness of the TTI-GAN model. The reason is that the TTI-GAN model can capture spatiotemporal correlations at a network level and model link TTDs explicitly even under extreme circumstances.

TABLE 2 Comparison of computational costs under the missing rate of 50% (Time unit: minute)

Method	ARIMA	k-NN	PPCA	DSAE	TTI-GAN
Time	20.65	10.12	8.09	255.90	223.71

Besides, Table 2 lists the time consumption of each imputation method under the missing rate of 50%. Obviously, the conventional methods (i.e., ARIMA, k-NN, and PPCA) consume less time than the deep learning-based methods (i.e., DSAE and TTI-GAN). This is not a surprise since deep learning-based methods have a higher computational complexity. It is also interesting to find that the TTI-GAN model requires less computation time compared to the DSAE model, which shows the merit of the proposed method.

In summary, compared with the conventional methods (i.e., ARIMA, k-NN, and PPCA), the deep learning-based TTI-GAN model can capture and make full use of the network-wide spatiotemporal correlations instead of utilizing the local information from the target link. Compared with DSAE, the TTI-GAN model can explore the traffic states depicted by link TTDs more effectively with the help of the semantic vectors and clustering labels provided by the Skip–Gram neural network model and GMM, respectively.

5 | CONCLUSIONS

This study proposes a TTI-GAN model to impute link travel times with GPS trajectories. Meanwhile, the Skip–Gram neural network model is incorporated to encode the spatial information of the underlying road network

and the temporal properties of link travel times in various time intervals. This helps the TTI-GAN model to take into account the spatiotemporal correlations among link travel times. The GMM is elaborated to explore the link traffic states and separate the link TTDs into various clusters, in which the corresponding best fine-tuned TTI-GANs are saved. Then, the numerical experiments are carried out with the link travel times drawn from GPS trajectories in the on-demand service platform of Didi Chuxing in Chengdu, China. Compared with the other four counterparts (i.e., ARIMA, k-NN, PPCA, and DSAE), the TTI-GAN model provides more accurate imputations of average travel times.

The contributions of this study are concluded as follows: (1) The TTI-GAN model can impute travel times for links without sufficient observations by modeling TTDs for links with rich data, which has the strong generalization and robustness, especially when the missing rate is high; (2) With the help of the semantic vectors from the Skip–Gram neural network model and the cluster labels from the GMM, the TTI-GAN model can well capture the spatiotemporal dependencies of link travel times at a network level. In summary, conditioned on the semantic vectors and cluster labels, the TTI-GAN model is well-suited to impute travel times under various missing data rates.

In the future, a mixture of data sources including radar data or other GPS data will be utilized to extend the usage of the proposed method. Moreover, due to the promising generative capability, it is encouraging to employ more GAN-based methods in the traffic field.

ACKNOWLEDGMENTS

This research has been supported by the National Key R&D Program of China (Grant No. 2018YFB1601300), the National Natural Science Foundation of China (Grant No. 71871227, 71871010, 61473114, 61973103), the Innovation Driven Plan of Central South University (Grant No. 20180016040002) and Fundamental Research Funds for the Henan Provincial Colleges and Universities in Henan University of Technology (Grant No. 2018RCJH16).

REFERENCES

Adeli, H., & Ghosh-Dastidar, S. (2004). Mesoscopic-wavelet freeway work zone flow and congestion feature extraction model. *Journal of Transportation Engineering*, *130*(1), 94–103.

Adeli, H., & Karim, A. (2005). *Wavelets in intelligent transportation systems*. Chichester, England: John Wiley & Sons.

Allison, P. D. (2001). *Missing data* (Vol. *136*). Thousand Oaks, CA: Sage Publications.

Bae, B., Kim, H., Lim, H., Liu, Y., Han, L. D., & Freeze, P. B. (2018). Missing data imputation for traffic flow speed using spatiotemporal cokriging. *Transportation Research Part C: Emerging Technologies*, *88*, 124–139.

Bie, Y., Xiong, X., Yan, Y., & Qu, X. (2020). Dynamic headway control for high-frequency bus line based on speed guidance and intersection signal adjustment. *Computer-Aided Civil and Infrastructure Engineering*, *35*(1), 4–25.

Chen, X., Duan, Y., Houthooft, R., Schulman, J., Sutskever, I., & Abbeel, P. (2016). Infogan: Interpretable representation learning by information maximizing generative adversarial nets. *Proceeding of the 30th International Conference on Neural Information Processing Systems*. Red Hook, NY: Curran Associates.

Chen, X., He, Z., & Sun, L. (2019). A Bayesian tensor decomposition approach for spatiotemporal traffic data imputation. *Transportation Research Part C: Emerging Technologies*, *98*, 73–84.

Chen, Y., Lv, Y., & Wang, F.-Y. (2019). Traffic flow imputation using parallel data and generative adversarial networks. *IEEE Transactions on Intelligent Transportation Systems*, *21*, 1624–1630. Retrieved from http://doi.org/10.1109/TITS.2019.2910295

Dobrushin, R. L. (1970). Prescribing a system of random variables by conditional distributions. *Theory of Probability & Its Applications*, *15*, 458–486.

Duan, Y. J., Lv, Y. S., Liu, C. J., & Wang, F. Y. (2016). An efficient realization of deep learning for traffic data imputation. *Transportation Research Part C: Emerging Technologies*, *72*, 168–181.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., … Bengio, Y. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, Vol. *27* (pp. 2672–2680). Red Hook, NY: Curran Associates.

Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. C. (2017). Improved training of wasserstein gans. *Advances in Neural Information Processing Systems*, Vol. *30* (pp. 5767–5777). Red Hook, NY: Curran Associates.

Haworth, J., & Cheng, T. (2012). Non-parametric regression for space-time forecasting under missing data. *Computers Environment and Urban Systems*, *36*(6), 538–550.

He, Z., Qi, G., Lu, L., & Chen Y. (2019). Network-wide identification of turn-level intersection congestion using only low-frequency probe vehicle data. *Transportation Research Part C: Emerging Technologies*, *108*, 320–339. http://doi.org/10.1016/j.trc.2019.10.001

He, Z., Zheng, L., Chen, P., & Guan, W. (2017). Mapping to cells: A simple method to extract traffic dynamics from probe vehicle data. *Computer-Aided Civil and Infrastructure Engineering*, *32*(3), 252–267.

Hunter, T., Abbeel, P., & Bayen, A. (2013). The path inference filter: Model-based low-latency map matching of probe vehicle data. *IEEE Transactions on Intelligent Transportation Systems*, *15*(2), 507–529.

Jiang, X., & Adeli, H. (2004a). Object-oriented model for freeway work zone capacity and queue delay estimation. *Computer-Aided Civil and Infrastructure Engineering*, *19*(2), 144–156.

Jiang, X., & Adeli, H. (2004b). Wavelet packet-autocorrelation function method for traffic flow pattern analysis. *Computer-Aided Civil and Infrastructure Engineering*, *19*(5), 324–337.

Karim, A., & Adeli, H. (2003). CBR model for freeway work zone traffic management. *Journal of Transportation Engineering*, *129*(2), 134–145.

Ku, W. C., Jagadeesh, G. R., Prakash, A., & Srikanthan, T. (2016). A clustering-based approach for data-driven imputation of missing traffic data. *2016 IEEE Forum on Integrated and Sustainable Transportation Systems (FISTS)*, 1–6. http://doi.org/10.1109/FISTS.2016.7552320

Laña, I., Olabarrieta, I. I., Vélez, M., & Del Ser, J. (2018). On the imputation of missing data for road traffic forecasting: New insights and novel techniques. *Transportation Research Part C: Emerging Technologies*, *90*, 18–33.

Li, L., Li, Y., & Li, Z. (2013). Efficient missing data imputing for traffic flow by considering temporal and spatial dependence. *Transportation Research Part C: Emerging Technologies*, *34*, 108–120.

Li, L., Su, X., Zhang, Y., Lin, Y., & Li, Z. (2015). Trend modeling for traffic time series analysis: An integrated study. *IEEE Transactions on Intelligent Transportation Systems*, *16*(6), 3430–3439.

Li, L., Zhang, J., Wang, Y., & Ran, B. (2018). Missing value imputation for traffic-related time series data based on a multi-view learning method. *IEEE Transactions on Intelligent Transportation Systems*, *20*(8), 2933–2943.

Li, Y., Li, Z., & Li, L. (2014). Missing traffic data: Comparison of imputation methods. *IET Intelligent Transport Systems*, *8*(1), 51–57.

Liang, Y., Cui, Z., Tian, Y., Chen, H., & Wang, Y. (2018). A deep generative adversarial architecture for network-wide spatial-temporal traffic-state estimation. *Transportation Research Record*, *2672*(45), 87–105.

Lin, Y., Dai, X., Li, L., & Wang, F.-Y. (2018). Pattern sensitive prediction of traffic flow based on generative adversarial framework. *IEEE Transactions on Intelligent Transportation Systems*, *20*(6), 2395–2400.

Liu, Y., Liu, Z., Vu, H. L., & Lyu, C. (2019). A spatio-temporal ensemble method for large-scale traffic state prediction. *Computer-Aided Civil and Infrastructure Engineering*, *35*(1), 26–44.

Lv, Y., Chen, Y., Li, L., & Wang, F.-Y. (2018). Generative adversarial networks for parallel transportation systems. *IEEE Intelligent Transportation Systems Magazine*, *10*(3), 4–10.

Lv, Y., Duan, Y., Kang, W., Li, Z., & Wang, F.-Y. (2014). Traffic flow prediction with big data: A deep learning approach. *IEEE Transactions on Intelligent Transportation Systems*, *16*(2), 865–873.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Proceedings of the 26th International Conference on Neural Information Processing Systems*, Vol. *2* (pp. 3111–3119). Red Hook, NY: Curran Associates.

Pearson, R. K. (2002). Outliers in process modeling and identification. *IEEE Transactions on Control Systems Technology*, *10*(1), 55–63.

Qu, L., Li, L., Zhang, Y., & Hu, J. (2009). PPCA-based missing data imputation for traffic flow volume: A systematical approach. *IEEE Transactions on Intelligent Transportation Systems*, *10*(3), 512–522.

Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv*:1511.06434.

Rafiei, M. H., & Adeli, H. (2017). A novel machine learning-based algorithm to detect damage in high-rise building structures. *The Structural Design of Tall and Special Buildings*, *26*(18), e1400.

Rafiei, M. H., & Adeli, H. (2018a). Novel machine-learning model for estimating construction costs considering economic variables and indexes. *Journal of Construction Engineering and Management*, *144*(12), 04018106.

Rafiei, M. H., & Adeli, H. (2018b). A novel unsupervised deep learning model for global and local health condition assessment of structures. *Engineering Structures*, *156*, 598–607.

Rafiei, M. H., Khushefati, W. H., Demirboga, R., & Adeli, H. (2017). Supervised deep restricted Boltzmann machine for estimation of concrete. *ACI Materials Journal*, *114*(2), 237–244.

Shang, Q., Yang, Z., Gao, S., & Tan, D. (2018). An imputation method for missing traffic data based on FCM optimized by PSO-SVR. *Journal of Advanced Transportation*, *2018*, 1–21.

Steinmetz, S. S., & Brownstone, D. (2005). Estimating commuters' "value of time" with noisy data: A multiple imputation approach. *Transportation Research Part B: Methodological*, *39*(10), 865–889.

Tang, J., Zhang, G., Wang, Y., Wang, H., & Liu, F. (2015). A hybrid approach to integrate fuzzy c-means based imputation method with genetic algorithm for missing traffic volume data estimation. *Transportation Research Part C: Emerging Technologies*, *51*, 29–40.

Van Lint, J., Hoogendoorn, S., & van Zuylen, H. J. (2005). Accurate freeway travel time prediction with state-space neural networks under missing data. *Transportation Research Part C: Emerging Technologies*, *13*(5-6), 347-369.

Vlahogianni, E. I., Karlaftis, M. G., & Golias, J. (2014). Short-term traffic forecasting: Where we are and where we're going. *Transportation Research Part C: Emerging Technologies*, *43*, 3–19.

Williams, C. K., & Rasmussen, C. E. (1996). Gaussian processes for regression. *Advances in Neural Information Processing Systems*, Vol. *8* (pp. 514–520). Red Hook, NY: Curran Associates.

Zhang, K., Liu, Z., & Zheng, L. (2019). Short-term prediction of passenger demand in multi-zone level: Temporal convolutional neural network with multi-task learning. *IEEE Transactions on Intelligent Transportation Systems*, *21*(4), 1480–1490.

Zhang, K., Zheng, L., Liu, Z., & Jia, N. (2019). A deep learning based multitask model for network-wide traffic speed predication. *Neurocomputing*, *396*, 438–450.

Zheng, L., Zhu, C., Zhu, N., He, T., Dong, N., & Huang, H. (2018). Feature selection-based approach for urban short-term travel speed prediction. *IET Intelligent Transport Systems*, *12*(6), 474–484.

Zhong, M., Lingras, P., & Sharma, S. (2004). Estimation of missing traffic counts using factor, genetic, neural, and regression techniques. *Transportation Research Part C: Emerging Technologies*, *12*(2), 139–166.

Zhuang, Y., Ke, R., & Wang, Y. (2019). Innovative method for traffic data imputation based on convolutional neural network. *IET Intelligent Transport Systems*, *13*(4), 605–613.

How to cite this article: Zhang K, He Z, Zheng L, Zhao L, Wu L. A generative adversarial network for travel times imputation using trajectory data. *Comput Aided Civ Inf*. 2021;36:197–212. https://doi.org/10.1111/mice.12595

APPENDIX

TABLE A1 Hyperparameters of the generator G and discriminator network

Generator G	Layer (type)	Parameters	Discriminator	Layer (type)	Parameters
	Input (*Linear*)	(*N, H*)		Input (*Linear*)	(N,H_T)
	Layer 1 (*BatchNorm1d*)	ϵ = 1e-5		Layer 1 (*Conv1d*)	filter (3, 1, 1)
	Layer 2 (*Upsample*)	factor = 2		Layer 2 (*LeakyReLU*)	alpha = 0.1
	Layer 3 (*Conv1d*)	filter (3, 1, 1)		Layer 3 (*Dropout*)	p = 0.25
	Layer 4 (*BatchNorm1d*)	ϵ = 1e-5		Layer 4 (*BatchNorm1d*)	ϵ = 1e-5
	Layer 5 (*LeakyReLU*)	alpha = 0.25		Layer 5 (*Conv1d*)	filter (3, 1, 1)
	Layer 6 (*Upsample*)	factor = 2		Layer 6 (*LeakyReLU*)	alpha = 0.25
	Layer 7 (*Conv1d*)	filter (5, 1, 1)		Layer 7 (*Dropout*)	p = 0.25
	Output (*Linear, Tanh*)	(N,H_T)		Layer 8 (*BatchNorm1d*)	ϵ = 1e-5
				Discriminator D_G (*Linear, Tanh*)	(*N, 1*)
				Recognition R_{L_N} (*Linear, Tanh*)	(N,H_{L_N})
				Recognition R_{L_C} (*Linear, Tanh*)	(N,H_{L_C})
				Recognition R_{C_1} (*Linear*)	(N,H_{C_1})
				Recognition R_{C_2} (*Linear*)	(N,H_{C_2})

Note: H_{L_N}, H_{L_C}, H_{C_1}, and H_{C_2} are the dimensions of $\boldsymbol{L}_N$, $\boldsymbol{L}_C$, $\boldsymbol{C}_1$, and $\boldsymbol{C}_2$, respectively.